

Lead AI Engineer Take-Home Assignment

Hi,

As the next step for the Lead AI Engineer role, please complete this short take-home assignment.

The goal is not to build a polished product. We want to understand how you design, implement, evaluate, and use modern AI-assisted engineering tools.

You may use Cursor, Claude Code, Codex, GitHub Copilot, or similar tools. We expect you to review, test, and explain the final solution yourself.

Task

Build a small AI learning assistant backend that answers student questions using the provided PDF book:

- `A Quick Algebra Review.pdf`

Your service should:

- ingest the PDF
- retrieve relevant context from the book
- answer student questions using an LLM
- return source references, preferably page number and section heading
- handle unsupported, unsafe, or academic dishonesty requests
- log basic quality/evaluation signals
- include tests or an evaluation script

Required API

Expose at minimum:

- `GET /health`
- `POST /ask`
- `POST /ingest`

Example `/ask` request:

```
{
  "student_id": "student-123",
  "question": "How do I solve  $4x - 7 = 21$ ?",
  "grade_level": "grade_8"
}
```

Example response shape:

```
{
  "request_id": "req-001",
  "answer": "Add 7 to both sides to get  $4x = 28$ . Then divide both sides by 4, so  $x = 7$ .",
  "sources": [
    {
      "source_id": "A Quick Algebra Review.pdf#page=4",
      "title": "Solving Equations",
      "page": 4,
      "excerpt": "..."
    }
  ],
  "safety_status": "answered"
}
```

Expectations

The PDF is mostly digital text, but it includes textbook-style extraction challenges such as formulas, examples, page breaks, and layout noise. Preserve page references and document any limitations.

The assistant should:

- avoid inventing citations
- say when the book does not contain enough information
- refuse or redirect unsafe/inappropriate requests
- be concise and student-friendly

Python, FastAPI, and a framework such as LangGraph are required for this test. You should use embeddings, hybrid search, reranking, or a vector database. Choose a practical approach and explain the trade-off.

Include a mock or deterministic mode for tests if your normal path requires an external LLM API key. Do not commit secrets. Do not build a UI.

Deliverables

Submit a zip file or repository link containing:

- README.md - setup, configuration, run commands, and example requests

- ARCHITECTURE.md - design choices, trade-offs, PDF ingestion approach, failure modes, and scaling approach
- AI_USAGE.md - how you used AI coding tools
- source code
- tests or an evaluation script

Your AI_USAGE.md should briefly cover:

- which AI tools you used
- what you asked them to do
- what you accepted or rejected
- one example where AI output was wrong or incomplete
- how you verified the final implementation
- what you would improve with more time

Time Expectation

Please spend no more than 4-6 hours.

We care more about clarity, judgment, and engineering quality than completeness. In the follow-up interview, we will discuss your design, review parts of the code, and ask you to extend or modify the solution live.